

SQRT Decomposition

Motivação

- Fonte de implementações, figuras, problemas, etc:
<https://usaco.guide/plat/sqrt?lang=cpp>
- Ideia: Dividir dados em blocos de tamanho $\sqrt{N}$ de modo a acelerar operações
- Pasta com material no github: <https://github.com/sallesviana/INF333-public/>

Motivação

- Motivação: <https://usaco.guide/plat/sqrt?lang=cpp>
 - Array com 10^5 elementos.
 - 10^5 consultas do tipo:
 - i. 2: Soma de intervalo
 - ii. 1: Trocar valor de elemento
 - Ideias?

3 2 4 5 1 1 5 3	14
2 1 4	2
2 5 6	11
1 3 1	
2 1 4	

Motivação

- Motivação: <https://usaco.guide/plat/sqrt?lang=cpp>
 - Array com 10^5 elementos.
 - 10^5 consultas do tipo:
 - i. 1: Soma de intervalo
 - ii. 2: Trocar valor de elemento
 - Ideias?
 - i. Seg-tree, por exemplo, resolve facilmente.
 - ii. Outra idéia: SQRT Decomposition

3 2 4 5 1 1 5 3	14
2 1 4	2
2 5 6	11
1 3 1	
2 1 4	

SQRT Decomposition

- Para facilitar, supor que consulta = soma(0,i) (obviamente, podemos usar isso para calcular somas de um intervalo).
- Dividir array em $\sqrt{N}$ blocos de tamanho $\sqrt{N}$
- Teremos para cada bloco a soma dele.
- Custo de atualizar um valor:

3 2 4 5 1 1 5 3	14
2 1 4	2
2 5 6	11
1 3 1	
2 1 4	

	3	2	4	5	1	1	5	3
soma bloco	9			7			8	

SQRT Decomposition

- Para facilitar, supor que consulta = soma(0,i) (obviamente, podemos usar isso para calcular somas de um intervalo.
- Dividir array em $\sqrt{N}$ blocos de tamanho $\sqrt{N}$
- Teremos para cada bloco a soma dele.
- Custo de atualizar um valor: $O(1)$
 - Basta trocar o valor e adicionar à soma.

3 2 4 5 1 1 5 3	14
2 1 4	2
2 5 6	11
1 3 1	
2 1 4	

	3	2	4	5	1	1	5	3
soma bloco	9			7			8	

	3	2	4	5	3	1	5	3
soma bloco	9			9			8	

SQRT Decomposition

- Para facilitar, supor que consulta = soma(0,i) (obviamente, podemos usar isso para calcular somas de um intervalo).
- Dividir array em $\sqrt{N}$ blocos de tamanho $\sqrt{N}$
- Teremos para cada bloco a soma dele.
- Custo de consulta = soma(0,i):

3 2 4 5 1 1 5 3	14
2 1 4	2
2 5 6	11
1 3 1	
2 1 4	

	3	2	4	5	1	1	5	3
soma bloco	9			7			8	

SQRT Decomposition

- Para facilitar, supor que consulta = soma(0,i) (obviamente, podemos usar isso para calcular somas de um intervalo).
- Dividir array em $\sqrt{N}$ blocos de tamanho $\sqrt{N}$
- Teremos para cada bloco a soma dele.
- Custo de consulta = soma(0,i):
 - Enquanto não chegar no bloco de i, usa a soma do bloco
 - Chegando, percorre os elementos do bloco
 - Exemplo: somar intervalo [0,7]
 - Custo?

3 2 4 5 1 1 5 3	14
2 1 4	2
2 5 6	11
1 3 1	
2 1 4	

	3	2	4	5	1	1	5	3
soma bloco	9			7			8	

SQRT Decomposition

- Para facilitar, supor que consulta = soma(0,i) (obviamente, podemos usar isso para calcular somas de um intervalo).
- Dividir array em $\sqrt{N}$ blocos de tamanho $\sqrt{N}$
- Teremos para cada bloco a soma dele.
- Custo de consulta = soma(0,i):
 - Enquanto não chegar no bloco de i, usa a soma do bloco
 - Chegando, percorre os elementos do bloco
 - Exemplo: somar intervalo [0,7]
 - Custo? $O(\sqrt{N})$ blocos + $O(\sqrt{N})$ dentro do bloco de i

3 2 4 5 1 1 5 3	14
2 1 4	2
2 5 6	11
1 3 1	
2 1 4	

	3	2	4	5	1	1	5	3
soma bloco	9			7			8	

SQRT Decomposition

- Obs: em vez de dividir o array em $\sqrt{N}$ blocos de tamanho $\sqrt{N}$, podemos dividir em K blocos de tamanho N/K
 - O K pode ser $\sqrt{N}$ ou algum outro valor, calibrado dependendo do problema (pode ser que em um problema uma operação com complexidade $O(K)$ seja realizada mais vezes do que uma com complexidade $O(N/K)$ → calibrar com $K < \sqrt{N}$ nesse caso).

3 2 4 5 1 1 5 3	14
2 1 4	2
2 5 6	11
1 3 1	
2 1 4	

	3	2	4	5	1	1	5	3
soma bloco	9			7			8	

SQRT Decomposition

- Outra forma de decompor:
- Criar prefix sum
- Qual a soma $[1,3]$?
 - $\text{prefix}[4] - \text{prefix}[1]$

	0	1	2	3	4	5	6	7	8
v	3	2	4	5	1	1	5	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Outra forma de decompor:
- Criar prefix sum
- Qual a soma [1,3] ?
 - $\text{prefix}[4] - \text{prefix}[1]$
- Se tiver atualização da posição 2 para 7, teríamos que somar quanto ao resultado?
 - $7 - v[2] = 7 - 4 = 3$
- Em cada atualização, guardar valores a somar em um array de pares (pos, somar) e atualizar no array v.
- Exemplo:
 - trocar pos 2 para 7 $\rightarrow (2,3)$ (somar 3 à posição 2)

	0	1	2	3	4	5	6	7	8
v	3	2	7	5	1	1	5	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Outra forma de decompor:
- Criar prefix sum
- Qual a soma $[1,3]$?
 - $\text{prefix}[4] - \text{prefix}[1]$
- Se tiver atualização da posição 2 para 7, teríamos que somar quanto ao resultado?
 - $7 - v[2] = 7 - 4 = 3$
- Em cada atualização, guardar valores a somar em um array de pares (pos, somar) e atualizar no array v.
- Exemplo:
 - trocar pos 2 para 7 $\rightarrow (2,3)$ (somar 3 à posição 2)
 - trocar pos 0 para 1 $\rightarrow (0,-2)$
 -

	0	1	2	3	4	5	6	7	8
v	1	2	7	5	1	1	5	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Outra forma de decompor:
- Criar prefix sum
- Qual a soma [1,3] ?
 - $\text{prefix}[4] - \text{prefix}[1]$
- Se tiver atualização da posição 2 para 7, teríamos que somar quanto ao resultado?
 - $7 - v[2] = 7 - 4 = 3$
- Em cada atualização, guardar valores a somar em um array de pares (pos, somar) e atualizar no array v.
- Exemplo:
 - trocar pos 2 para 7 $\rightarrow (2,3)$ (somar 3 à posição 2)
 - trocar pos 0 para 1 $\rightarrow (0,-2)$
 - trocar pos 6 para 7 $\rightarrow (6,2)$

	0	1	2	3	4	5	6	7	8
v	1	2	7	5	1	1	7	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Outra forma de decompor:
- Criar prefix sum
- Qual a soma $[1,3]$?
 - $\text{prefix}[4] - \text{prefix}[1]$
- Se tiver atualização da posição 2 para 7, teríamos que somar quanto ao resultado?
 - $7 - v[2] = 7 - 4 = 3$
- Em cada atualização, guardar valores a somar em um array de pares (pos, somar) e atualizar no array v.
- Exemplo:
 - trocar pos 2 para 7 $\rightarrow (2,3)$ (somar 3 à posição 2)
 - trocar pos 0 para 1 $\rightarrow (0,-2)$
 - trocar pos 6 para 7 $\rightarrow (6,2)$
 - trocar pos 2 para 5 $\rightarrow (2,-2)$

	0	1	2	3	4	5	6	7	8
v	1	2	5	5	1	1	7	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Outra forma de decompor:
- Criar prefix sum
- Qual a soma $[1,3]$?
 - $\text{prefix}[4] - \text{prefix}[1]$
- Se tiver atualização da posição 2 para 7, teríamos que somar quanto ao resultado?
 - $7 - v[2] = 7 - 4 = 3$
- Em cada atualização, guardar valores a somar em um array de pares (pos, somar) e atualizar no array v.
- Exemplo:
 - trocar pos 2 para 7 $\rightarrow (2,3)$ (somar 3 à posição 2)
 - trocar pos 0 para 1 $\rightarrow (0,-2)$
 - trocar pos 6 para 7 $\rightarrow (6,2)$
 - trocar pos 2 para 5 $\rightarrow (2,-2)$
- Qual a soma de $[0,2]$?
 - $9 - 0 = 9$ (prefixo está desatualizado...)

	0	1	2	3	4	5	6	7	8
v	1	2	5	5	1	1	7	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Outra forma de decompor:
- Criar prefix sum
- Qual a soma [1,3] ?
 - $\text{prefix}[4] - \text{prefix}[1]$
- Se tiver atualização da posição 2 para 7, teríamos que somar quanto ao resultado?
 - $7 - v[2] = 7 - 4 = 3$
- Em cada atualização, guardar valores a somar em um array de pares (pos, somar) e atualizar no array v.
- Exemplo:
 - trocar pos 2 para 7 $\rightarrow (2,3)$ (somar 3 à posição 2)
 - trocar pos 0 para 1 $\rightarrow (0,-2)$
 - trocar pos 6 para 7 $\rightarrow (6,2)$
 - trocar pos 2 para 5 $\rightarrow (2,-2)$
- Qual a soma de [0,2] ?
 - $9 - 0 = 9$ (prefixo está desatualizado...)
 - $9 + 3 + -2 + -2 = 8$

	0	1	2	3	4	5	6	7	8
v	1	2	5	5	1	1	7	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Complexidade?

```
vector<pair<int, long long> > atualizacoes;  
for (int i = 0; i < q; i++) {  
    int t, l, r;  
    cin >> t >> l >> r;  
    if (t == 1) {  
        //sq.update(l - 1, r);  
        atualizacoes.push_back( make_pair(l-1, r-v[l-1]) );  
        v[l-1] = r;  
    } else { //calcular soma de l-1 até r-1  
        long long soma = prefix[r] - prefix[l-1];  
        //varre as atualizacoes corrigindo soma...  
        for(auto &[pos, valor]:atualizacoes) {  
            if( l-1 <= pos && pos <= r-1)  
                soma += valor;  
        }  
  
        cout << soma << "\n"; //uso r aqui, porque conta a  
    }  
}
```

	0	1	2	3	4	5	6	7	8
v	1	2	6	5	1	1	7	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Complexidade?
 - Atualizacao: $O(1)$
 - Consulta: $O(\sqrt{n})$
- N consultas/atualizacoes pode ficar $O(N^2)$
- Ideias para melhorar?

```
vector<pair<int, long long> > atualizacoes;  
for (int i = 0; i < q; i++) {  
    int t, l, r;  
    cin >> t >> l >> r;  
    if (t == 1) {  
        //sq.update(l - 1, r);  
        atualizacoes.push_back( make_pair(l-1, r-v[l-1]) );  
        v[l-1] = r;  
    } else { //calcular soma de l-1 até r-1  
        long long soma = prefix[r] - prefix[l-1];  
        //varre as atualizacoes corrigindo soma...  
        for(auto &[pos, valor]:atualizacoes) {  
            if( l-1 <= pos && pos <= r-1)  
                soma += valor;  
        }  
  
        cout << soma << "\n"; //uso r aqui, porque conta a  
    }  
}
```

	0	1	2	3	4	5	6	7	8
v	1	2	6	5	1	1	7	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Complexidade?
 - Atualizacao: $O(1)$
 - Consulta: $O(|atualizacoes|)$
- N consultas/atualizacoes pode ficar $O(N^2)$
- Solução:
 - Quando atualizações ficar com tamanho $O(\sqrt{N})$ (ou outro valor a calibrar) → recalculer a soma de prefixos e deixar vetor de atualizações vazio.
 - Consulta: $O(\sqrt{N})$
 - A cada $\sqrt{N}$ iterações → recalculer prefixo: $O(N \sqrt{N})$
 - Tempo total: $O(N \sqrt{N})$

```
vector<pair<int, long long> > atualizacoes;  
for (int i = 0; i < q; i++) {  
    int t, l, r;  
    cin >> t >> l >> r;  
    if (t == 1) {  
        //sq.update(l - 1, r);  
        atualizacoes.push_back( make_pair(l-1, r-v[l-1]) );  
        v[l-1] = r;  
    } else { //calcular soma de l-1 até r-1  
        long long soma = prefix[r] - prefix[l-1];  
        //varre as atualizacoes corrigindo soma...  
        for(auto &[pos, valor]:atualizacoes) {  
            if( l-1 <= pos && pos <= r-1)  
                soma += valor;  
        }  
  
        cout << soma << "\n"; //uso r aqui, porque conta a i  
    }  
}
```

	0	1	2	3	4	5	6	7	8
v	1	2	6	5	1	1	7	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Essa nova forma também é conhecida como blocking.
- A entrada é dividida em blocos.
- De tempos em tempos os blocos são processados e são feitas atualizações.
- Exercício: implemente o código para resolver esse problema (sem ver a solução do github)

```
vector<pair<int, long long> > atualizacoes;  
for (int i = 0; i < q; i++) {  
    int t, l, r;  
    cin >> t >> l >> r;  
    if (t == 1) {  
        //sq.update(l - 1, r);  
        atualizacoes.push_back( make_pair(l-1, r-v[l-1]) );  
        v[l-1] = r;  
    } else { //calcular soma de l-1 até r-1  
        long long soma = prefix[r] - prefix[l-1];  
        //varre as atualizacoes corrigindo soma...  
        for(auto &[pos, valor]:atualizacoes) {  
            if( l-1 <= pos && pos <= r-1)  
                soma += valor;  
        }  
  
        cout << soma << "\n"; //uso r aqui, porque conta a i  
    }  
}
```

	0	1	2	3	4	5	6	7	8
v	1	2	6	5	1	1	7	3	
prefix	0	3	5	9	14	15	16	21	24

SQRT Decomposition

- Outra técnica: combinar dois algoritmos diferentes.
- Ambos algoritmos resolvem o mesmo problema em $O(n^2)$
- Usando-se um algoritmo quando ele é melhor e o outro algoritmo quando o outro é melhor, obtemos tempo $O(n \sqrt{n})$

SQRT Decomposition

- Outro problema: https://atcoder.jp/contests/abc335/tasks/abc335_f
- N células cada uma com um valor $v[i] \geq 0$.
- Célula 1 tem um item.
- Quando item passa em uma célula (inclusive a 1) $\rightarrow$ ela fica marcada.
- A seguinte operação pode ser realizada qualquer quantidade de vezes:
 - Se item está na célula i , você pode escolher um inteiro não negativo X e mover a peça para posição $v[i]*X$
- De quantas formas diferentes podemos marcar o vetor? (%998244353)

SQRT Decomposition

- A seguinte operação pode ser realizada qualquer quantidade de vezes:
 - Se item está na célula i , você pode escolher um inteiro não negativo X e mover a peça para posição $v[i]*X$
- Exemplo:
 - Célula 1 (não faz operação)
 - Célula 1,2
 - Célula 1,2,4
 - Célula 1,2,4,5 (exemplo: $1, 1+1*1, 2+2*1, 4+1*1$)
 - Célula 1,3
 - Célula 1,4
 - Célula 1,4,5
 - Célula 1,5
- Como resolver?

Célula	1	2	3	4	5
$v[]$	1	2	3	1	1

SQRT Decomposition

- A seguinte operação pode ser realizada qualquer quantidade de vezes:
 - Se item está na célula i , você pode escolher um inteiro não negativo X e mover a peça para posição $v[i]*X$
- Exemplo:
 - Célula 1 (não faz operação)
 - Célula 1,2
 - Célula 1,2,4
 - Célula 1,2,4,5 (exemplo: $1, 1+1*1, 2+2*1, 4+1*1$)
 - Célula 1,3
 - Célula 1,4
 - Célula 1,4,5
 - Célula 1,5
- Como resolver? dica: PD

Célula	1	2	3	4	5
$v[]$	1	2	3	1	1

SQRT Decomposition

- A seguinte operação pode ser realizada qualquer quantidade de vezes:
 - Se item está na célula i , você pode escolher um inteiro não negativo X e mover a peça para posição $v[i]*X$
- Exemplo:
 - Célula 1 (não faz operação)
 - Célula 1,2
 - Célula 1,2,4
 - Célula 1,2,4,5 (exemplo: $1, 1+1*1, 2+2*1, 4+1*1$)
 - Célula 1,3
 - Célula 1,4
 - Célula 1,4,5
 - Célula 1,5
- Como resolver? dica: PD
 - $pd[i]$ = de quantas formas consigo pintar a partir de i , supondo que i está pintado

Célula	1	2	3	4	5
$v[]$	1	2	3	1	1
pd					1

SQRT Decomposition

- A seguinte operação pode ser realizada qualquer quantidade de vezes:
 - Se item está na célula i , você pode escolher um inteiro não negativo X e mover a peça para posição $v[i]*X$
- Exemplo:
 - Célula 1 (não faz operação)
 - Célula 1,2
 - Célula 1,2,4
 - Célula 1,2,4,5 (exemplo: $1, 1+1*1, 2+2*1, 4+1*1$)
 - Célula 1,3
 - Célula 1,4
 - Célula 1,4,5
 - Célula 1,5
- Como resolver? dica: PD
 - $pd[i]$ = de quantas formas consigo pintar a partir de i , supondo que i está pintado

Célula	1	2	3	4	5
$v[]$	1	2	3	1	1
pd				2	1

SQRT Decomposition

- A seguinte operação pode ser realizada qualquer quantidade de vezes:
 - Se item está na célula i , você pode escolher um inteiro não negativo X e mover a peça para posição $v[i]*X$
- Exemplo:
 - Célula 1 (não faz operação)
 - Célula 1,2
 - Célula 1,2,4
 - Célula 1,2,4,5 (exemplo: $1, 1+1*1, 2+2*1, 4+1*1$)
 - Célula 1,3
 - Célula 1,4
 - Célula 1,4,5
 - Célula 1,5
- Como resolver? dica: PD
 - $pd[i]$ = de quantas formas consigo pintar a partir de i , supondo que i está pintado

Célula	1	2	3	4	5
$v[]$	1	2	3	1	1
pd			1	2	1

SQRT Decomposition

- A seguinte operação pode ser realizada qualquer quantidade de vezes:
 - Se item está na célula i , você pode escolher um inteiro não negativo X e mover a peça para posição $v[i]*X$
- Exemplo:
 - Célula 1 (não faz operação)
 - Célula 1,2
 - Célula 1,2,4
 - Célula 1,2,4,5 (exemplo: $1, 1+1*1, 2+2*1, 4+1*1$)
 - Célula 1,3
 - Célula 1,4
 - Célula 1,4,5
 - Célula 1,5
- Como resolver? dica: PD
 - $pd[i]$ = de quantas formas consigo pintar a partir de i , supondo que i está pintado

Célula	1	2	3	4	5
$v[]$	1	2	3	1	1
pd		3	1	2	1

SQRT Decomposition

- A seguinte operação pode ser realizada qualquer quantidade de vezes:
 - Se item está na célula i , você pode escolher um inteiro não negativo X e mover a peça para posição $v[i]*X$

- Exemplo:

- Célula 1 (não faz operação)
- Célula 1,2
- Célula 1,2,4
- Célula 1,2,4,5 (exemplo: 1, $1+1*1$, $2+2*1$, $4+1*1$)
- Célula 1,3
- Célula 1,4
- Célula 1,4,5
- Célula 1,5

Célula	1	2	3	4	5
$v[]$	1	2	3	1	1
pd	8	3	1	2	1

- Como resolver? dica: PD

- $pd[i]$ = de quantas formas consigo pintar a partir de i , supondo que i está pintado
- $pd[i] = ?$

SQRT Decomposition

- A seguinte operação pode ser realizada qualquer quantidade de vezes:
 - Se item está na célula i , você pode escolher um inteiro não negativo X e mover a peça para posição $v[i]*X$

- Exemplo:

- Célula 1 (não faz operação)
- Célula 1,2
- Célula 1,2,4
- Célula 1,2,4,5 (exemplo: 1, $1+1*1$, $2+2*1$, $4+1*1$)
- Célula 1,3
- Célula 1,4
- Célula 1,4,5
- Célula 1,5

Célula	1	2	3	4	5
$v[]$	1	2	3	1	1
pd	8	3	1	2	1

- Como resolver? dica: PD

- $pd[i]$ = de quantas formas consigo pintar a partir de i , supondo que i está pintado
- $pd[i] = 1 + \text{somatorio}(pd[i + x * v[i]], \text{ enquanto } i + x*v[i] \leq n)$ (ir tirando módulo...)
- Resposta = $pd[1]$

SQRT Decomposition

- Como resolver? dica: PD
 - $pd[i]$ = de quantas formas consigo pintar a partir de i , supondo que i está pintado
 - $pd[i] = 1 + \text{somatorio}(pd[i + x * v[i]], \text{ enquanto } i + x * v[i] \leq n)$ (ir tirando módulo...)
 - Resposta = $pd[1]$

Célula	1	2	3	4	5
v[]	1	2	3	1	1
pd	8	3	1	2	1

```
vector<ull> dp(n,1);
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado :
    //e que já calculei em dp as respostas para todos quadrados da frente?
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;
    for(int x=1;i+x*elem < n;x++) { //podemos usar x= 0,1,2,3,... do elemento
        ans = (ans + dp[i+x*elem])%998244353;
    }
    dp[i] = ans;
}

cout << dp[0] << endl;
```

SQRT Decomposition

- Como resolver? dica: PD
 - $pd[i]$ = de quantas formas consigo pintar a partir de i , supondo que i está pintado
 - $pd[i] = 1 + \text{somatorio}(pd[i + x * v[j]], \text{ enquanto } i + x * v[j] \leq n)$ (ir tirando módulo...)
 - Resposta = $pd[1]$
- Excelente!

Célula	1	2	3	4	5
v[]	1	2	3	1	1
pd	8	3	1	2	1

```
vector<ull> dp(n,1);
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado :
    //e que já calculei em dp as respostas para todos quadrados da frente?
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;
    for(int x=1;i+x*elem < n;x++) { //podemos usar x= 0,1,2,3,... do elemento
        ans = (ans + dp[i+x*elem])%998244353;
    }
    dp[i] = ans;
}

cout << dp[0] << endl;
```

SQRT Decomposition

- Como resolver? dica: PD
 - $pd[i]$ = de quantas formas consigo pintar a partir de i , supondo que i está pintado
 - $pd[i] = 1 + \text{somatorio}(pd[i + x * v[j]], \text{ enquanto } i + x*v[j] \leq n)$ (ir tirando módulo...)
 - Resposta = $pd[1]$
- Excelente forma de ganhar TLE!
 - Quando elem é pequeno, o for interno é caro
 - $O(N^2)$
 - $N = 2*10^5$

Célula	1	2	3	4	5
v[]	1	2	3	1	1
pd	8	3	1	2	1

```
vector<ull> dp(n,1);
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado :
    //e que já calculei em dp as respostas para todos quadrados da frente?
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;
    for(int x=1;i+x*elem < n;x++) { //podemos usar x= 0,1,2,3,... do elemento
        ans = (ans + dp[i+x*elem])%998244353;
    }
    dp[i] = ans;
}

cout << dp[0] << endl;
```

SQRT Decomposition

- E se guardarmos as somas parciais que saltam de elem em elem até o momento?
- Vamos reescrever o código para tentar descobrir como o somatório está sendo feito...

```
14
15     for(int elem=1;elem<=4;elem++) {
16         for(int i=n-1;i>=0;i--) {
17             cout << "Se i= " << i << " e elem = " << elem << ", vamos somar: ";
18             //somatório de (i + x*elem)
19             for(int x=1; i+x*elem < n ; x++) { //posso multiplicar o elem por 1,2,3
20                 cout << i+x*elem << " ";
21             }
22             cout << endl;
23         }
24         cout << endl;
25     }
26     return 0;
27 }
```

```
vector<ull> dp(n,1);
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado :
    //e que já calculei em dp as respostas para todos quadrados da frente?
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;
    for(int x=1;i+x*elem < n;x++) { //podemos usar x= 0,1,2,3,... do elemento
        ans = (ans + dp[i+x*elem])%998244353;
    }
    dp[i] = ans;
}

cout << dp[0] << endl;
```

Se i= 9 e elem = 1, vamos somar:
 Se i= 8 e elem = 1, vamos somar: 9
 Se i= 7 e elem = 1, vamos somar: 8 9
 Se i= 6 e elem = 1, vamos somar: 7 8 9
 Se i= 5 e elem = 1, vamos somar: 6 7 8 9
 Se i= 4 e elem = 1, vamos somar: 5 6 7 8 9
 Se i= 3 e elem = 1, vamos somar: 4 5 6 7 8 9
 Se i= 2 e elem = 1, vamos somar: 3 4 5 6 7 8 9
 Se i= 1 e elem = 1, vamos somar: 2 3 4 5 6 7 8 9
 Se i= 0 e elem = 1, vamos somar: 1 2 3 4 5 6 7 8 9

Se i= 9 e elem = 2, vamos somar:
 Se i= 8 e elem = 2, vamos somar:
 Se i= 7 e elem = 2, vamos somar: 9
 Se i= 6 e elem = 2, vamos somar: 8
 Se i= 5 e elem = 2, vamos somar: 7 9
 Se i= 4 e elem = 2, vamos somar: 6 8
 Se i= 3 e elem = 2, vamos somar: 5 7 9
 Se i= 2 e elem = 2, vamos somar: 4 6 8
 Se i= 1 e elem = 2, vamos somar: 3 5 7 9
 Se i= 0 e elem = 2, vamos somar: 2 4 6 8

Se i= 9 e elem = 3, vamos somar:
 Se i= 8 e elem = 3, vamos somar:
 Se i= 7 e elem = 3, vamos somar:
 Se i= 6 e elem = 3, vamos somar: 9
 Se i= 5 e elem = 3, vamos somar: 8
 Se i= 4 e elem = 3, vamos somar: 7
 Se i= 3 e elem = 3, vamos somar: 6 9
 Se i= 2 e elem = 3, vamos somar: 5 8
 Se i= 1 e elem = 3, vamos somar: 4 7
 Se i= 0 e elem = 3, vamos somar: 3 6 9

Se i= 9 e elem = 4, vamos somar:
 Se i= 8 e elem = 4, vamos somar:
 Se i= 7 e elem = 4, vamos somar:
 Se i= 6 e elem = 4, vamos somar:
 Se i= 5 e elem = 4, vamos somar: 9
 Se i= 4 e elem = 4, vamos somar: 8
 Se i= 3 e elem = 4, vamos somar: 7
 Se i= 2 e elem = 4, vamos somar: 6
 Se i= 1 e elem = 4, vamos somar: 5 9
 Se i= 0 e elem = 4, vamos somar: 4 8

- Se elem for 1, temos basicamente que guardar as somas até o momento...
 - $i = 9 \rightarrow dp[9] = 1$
 - $i = 8 \rightarrow dp[8] = 1 + dp[9]$
 - $i = 7 \rightarrow dp[7] = 1 + dp[8] + dp[9]$
 - $i = 6 \rightarrow dp[6] = 1 + dp[7] + dp[8] + dp[9]$
 - $dp[i] = 1 + dp[i-1] = 1 + somaAnterior$
- Se elem for 2:
 - $i = 9 \rightarrow dp[9] = 1$
 - $i = 8 \rightarrow dp[8] = 1$
 - $i = 7 \rightarrow dp[7] = 1 + ?$
 - $i = 6 \rightarrow dp[6] = 1 + ?$

Se i= 9 e elem = 1, vamos somar:
 Se i= 8 e elem = 1, vamos somar: 9
 Se i= 7 e elem = 1, vamos somar: 8 9
 Se i= 6 e elem = 1, vamos somar: 7 8 9
 Se i= 5 e elem = 1, vamos somar: 6 7 8 9
 Se i= 4 e elem = 1, vamos somar: 5 6 7 8 9
 Se i= 3 e elem = 1, vamos somar: 4 5 6 7 8 9
 Se i= 2 e elem = 1, vamos somar: 3 4 5 6 7 8 9
 Se i= 1 e elem = 1, vamos somar: 2 3 4 5 6 7 8 9
 Se i= 0 e elem = 1, vamos somar: 1 2 3 4 5 6 7 8 9

Se i= 9 e elem = 2, vamos somar:
 Se i= 8 e elem = 2, vamos somar:
 Se i= 7 e elem = 2, vamos somar: 9
 Se i= 6 e elem = 2, vamos somar: 8
 Se i= 5 e elem = 2, vamos somar: 7 9
 Se i= 4 e elem = 2, vamos somar: 6 8
 Se i= 3 e elem = 2, vamos somar: 5 7 9
 Se i= 2 e elem = 2, vamos somar: 4 6 8
 Se i= 1 e elem = 2, vamos somar: 3 5 7 9
 Se i= 0 e elem = 2, vamos somar: 2 4 6 8

Se i= 9 e elem = 3, vamos somar:
 Se i= 8 e elem = 3, vamos somar:
 Se i= 7 e elem = 3, vamos somar:
 Se i= 6 e elem = 3, vamos somar: 9
 Se i= 5 e elem = 3, vamos somar: 8
 Se i= 4 e elem = 3, vamos somar: 7
 Se i= 3 e elem = 3, vamos somar: 6 9
 Se i= 2 e elem = 3, vamos somar: 5 8
 Se i= 1 e elem = 3, vamos somar: 4 7
 Se i= 0 e elem = 3, vamos somar: 3 6 9

Se i= 9 e elem = 4, vamos somar:
 Se i= 8 e elem = 4, vamos somar:
 Se i= 7 e elem = 4, vamos somar:
 Se i= 6 e elem = 4, vamos somar:
 Se i= 5 e elem = 4, vamos somar: 9
 Se i= 4 e elem = 4, vamos somar: 8
 Se i= 3 e elem = 4, vamos somar: 7
 Se i= 2 e elem = 4, vamos somar: 6
 Se i= 1 e elem = 4, vamos somar: 5 9
 Se i= 0 e elem = 4, vamos somar: 4 8

- Se elem for 1, temos basicamente que guardar as somas até o momento...
 - $i = 9 \rightarrow dp[9] = 1$
 - $i = 8 \rightarrow dp[8] = 1 + dp[9]$
 - $i = 7 \rightarrow dp[7] = 1 + dp[8] + dp[9]$
 - $i = 6 \rightarrow dp[6] = 1 + dp[7] + dp[8] + dp[9]$
 - $dp[i] = 1 + dp[i-1] = 1 + somaAnterior$
- Se elem for 2:
 - $i = 9 \rightarrow dp[9] = 1$
 - $i = 8 \rightarrow dp[8] = 1$
 -
 - $i = 7 \rightarrow dp[7] = 1 + dp[9]$
 - $i = 6 \rightarrow dp[6] = 1 + dp[8]$
 -
 - $i = 5 \rightarrow dp[5] = 1 + dp[7] + dp[9]$
 - $i = 4 \rightarrow dp[4] = 1 + dp[6] + dp[8]$
 -
 - $dp[i] = 1 + dp[i-2] = 1 + somaAnterior[0]$, se i par
 - $dp[i] = 1 + dp[i-2] = 1 + somaAnterior[1]$, se i ímpar
 -
 - $dp[i] = 1 + dp[i-2] = 1 + somaAnterior[i\%2]$
 - Ou seja, temos que guardar 2 somas para elem=2

Se i= 9 e elem = 1, vamos somar:
Se i= 8 e elem = 1, vamos somar: 9
Se i= 7 e elem = 1, vamos somar: 8 9
Se i= 6 e elem = 1, vamos somar: 7 8 9
Se i= 5 e elem = 1, vamos somar: 6 7 8 9
Se i= 4 e elem = 1, vamos somar: 5 6 7 8 9
Se i= 3 e elem = 1, vamos somar: 4 5 6 7 8 9
Se i= 2 e elem = 1, vamos somar: 3 4 5 6 7 8 9
Se i= 1 e elem = 1, vamos somar: 2 3 4 5 6 7 8 9
Se i= 0 e elem = 1, vamos somar: 1 2 3 4 5 6 7 8 9

Se i= 9 e elem = 2, vamos somar:
Se i= 8 e elem = 2, vamos somar:
Se i= 7 e elem = 2, vamos somar: 9
Se i= 6 e elem = 2, vamos somar: 8
Se i= 5 e elem = 2, vamos somar: 7 9
Se i= 4 e elem = 2, vamos somar: 6 8
Se i= 3 e elem = 2, vamos somar: 5 7 9
Se i= 2 e elem = 2, vamos somar: 4 6 8
Se i= 1 e elem = 2, vamos somar: 3 5 7 9
Se i= 0 e elem = 2, vamos somar: 2 4 6 8

Se i= 9 e elem = 3, vamos somar:
Se i= 8 e elem = 3, vamos somar:
Se i= 7 e elem = 3, vamos somar:
Se i= 6 e elem = 3, vamos somar: 9
Se i= 5 e elem = 3, vamos somar: 8
Se i= 4 e elem = 3, vamos somar: 7
Se i= 3 e elem = 3, vamos somar: 6 9
Se i= 2 e elem = 3, vamos somar: 5 8
Se i= 1 e elem = 3, vamos somar: 4 7
Se i= 0 e elem = 3, vamos somar: 3 6 9

Se i= 9 e elem = 4, vamos somar:
Se i= 8 e elem = 4, vamos somar:
Se i= 7 e elem = 4, vamos somar:
Se i= 6 e elem = 4, vamos somar:
Se i= 5 e elem = 4, vamos somar: 9
Se i= 4 e elem = 4, vamos somar: 8
Se i= 3 e elem = 4, vamos somar: 7
Se i= 2 e elem = 4, vamos somar: 6
Se i= 1 e elem = 4, vamos somar: 5 9
Se i= 0 e elem = 4, vamos somar: 4 8

- Se elem for 3:
 - Temos que guardar 3 somatórios...
 - $dp[i] = 1 + somaAnterior[i \% 3]$
 - $somaAnterior[i \% 3] = dp[i]$ //fazemos isso depois para usar na próxima iteração...
- Em uma iteração, o valor que vamos somar depende do elemento que está na posição do vetor...
 - Exemplo, na iteração 1 (negrito)....
 - $somaAnterior[elem][]$ terá as somas anteriores considerando saltos de elem em elem..
 - $dp[i] = 1 + somaAnterior[?][?]$

Se i= 9 e elem = 1, vamos somar:
Se i= 8 e elem = 1, vamos somar: 9
Se i= 7 e elem = 1, vamos somar: 8 9
Se i= 6 e elem = 1, vamos somar: 7 8 9
Se i= 5 e elem = 1, vamos somar: 6 7 8 9
Se i= 4 e elem = 1, vamos somar: 5 6 7 8 9
Se i= 3 e elem = 1, vamos somar: 4 5 6 7 8 9
Se i= 2 e elem = 1, vamos somar: 3 4 5 6 7 8 9
Se i= 1 e elem = 1, vamos somar: 2 3 4 5 6 7 8 9
Se i= 0 e elem = 1, vamos somar: 1 2 3 4 5 6 7 8 9

Se i= 9 e elem = 2, vamos somar:
Se i= 8 e elem = 2, vamos somar:
Se i= 7 e elem = 2, vamos somar: 9
Se i= 6 e elem = 2, vamos somar: 8
Se i= 5 e elem = 2, vamos somar: 7 9
Se i= 4 e elem = 2, vamos somar: 6 8
Se i= 3 e elem = 2, vamos somar: 5 7 9
Se i= 2 e elem = 2, vamos somar: 4 6 8
Se i= 1 e elem = 2, vamos somar: 3 5 7 9
Se i= 0 e elem = 2, vamos somar: 2 4 6 8

Se i= 9 e elem = 3, vamos somar:
Se i= 8 e elem = 3, vamos somar:
Se i= 7 e elem = 3, vamos somar:
Se i= 6 e elem = 3, vamos somar: 9
Se i= 5 e elem = 3, vamos somar: 8
Se i= 4 e elem = 3, vamos somar: 7
Se i= 3 e elem = 3, vamos somar: 6 9
Se i= 2 e elem = 3, vamos somar: 5 8
Se i= 1 e elem = 3, vamos somar: 4 7
Se i= 0 e elem = 3, vamos somar: 3 6 9

Se i= 9 e elem = 4, vamos somar:
Se i= 8 e elem = 4, vamos somar:
Se i= 7 e elem = 4, vamos somar:
Se i= 6 e elem = 4, vamos somar:
Se i= 5 e elem = 4, vamos somar: 9
Se i= 4 e elem = 4, vamos somar: 8
Se i= 3 e elem = 4, vamos somar: 7
Se i= 2 e elem = 4, vamos somar: 6
Se i= 1 e elem = 4, vamos somar: 5 9
Se i= 0 e elem = 4, vamos somar: 4 8

- Se elem for 3:
 - Temos que guardar 3 somatórios...
 - $dp[i] = 1 + somaAnterior[i \% 3]$
 - $somaAnterior[i \% 3] = dp[i]$ //fazemos isso depois para usar na próxima iteração...
- Em uma iteração, o valor que vamos somar depende do elemento que está na posição do vetor...
 - Exemplo, na iteração 1....
 - $somaAnterior[elem][]$ terá as somas anteriores considerando saltos de elem em elem..
 - $dp[i] = 1 + somaAnterior[elem][?]$

Se i= 9 e elem = 1, vamos somar:
Se i= 8 e elem = 1, vamos somar: 9
Se i= 7 e elem = 1, vamos somar: 8 9
Se i= 6 e elem = 1, vamos somar: 7 8 9
Se i= 5 e elem = 1, vamos somar: 6 7 8 9
Se i= 4 e elem = 1, vamos somar: 5 6 7 8 9
Se i= 3 e elem = 1, vamos somar: 4 5 6 7 8 9
Se i= 2 e elem = 1, vamos somar: 3 4 5 6 7 8 9
Se i= 1 e elem = 1, vamos somar: 2 3 4 5 6 7 8 9
Se i= 0 e elem = 1, vamos somar: 1 2 3 4 5 6 7 8 9

Se i= 9 e elem = 2, vamos somar:
Se i= 8 e elem = 2, vamos somar:
Se i= 7 e elem = 2, vamos somar: 9
Se i= 6 e elem = 2, vamos somar: 8
Se i= 5 e elem = 2, vamos somar: 7 9
Se i= 4 e elem = 2, vamos somar: 6 8
Se i= 3 e elem = 2, vamos somar: 5 7 9
Se i= 2 e elem = 2, vamos somar: 4 6 8
Se i= 1 e elem = 2, vamos somar: 3 5 7 9
Se i= 0 e elem = 2, vamos somar: 2 4 6 8

Se i= 9 e elem = 3, vamos somar:
Se i= 8 e elem = 3, vamos somar:
Se i= 7 e elem = 3, vamos somar:
Se i= 6 e elem = 3, vamos somar: 9
Se i= 5 e elem = 3, vamos somar: 8
Se i= 4 e elem = 3, vamos somar: 7
Se i= 3 e elem = 3, vamos somar: 6 9
Se i= 2 e elem = 3, vamos somar: 5 8
Se i= 1 e elem = 3, vamos somar: 4 7
Se i= 0 e elem = 3, vamos somar: 3 6 9

Se i= 9 e elem = 4, vamos somar:
Se i= 8 e elem = 4, vamos somar:
Se i= 7 e elem = 4, vamos somar:
Se i= 6 e elem = 4, vamos somar:
Se i= 5 e elem = 4, vamos somar: 9
Se i= 4 e elem = 4, vamos somar: 8
Se i= 3 e elem = 4, vamos somar: 7
Se i= 2 e elem = 4, vamos somar: 6
Se i= 1 e elem = 4, vamos somar: 5 9
Se i= 0 e elem = 4, vamos somar: 4 8

- Se elem for 3:
 - Temos que guardar 3 somatórios...
 - $dp[i] = 1 + somaAnterior[i \% 3]$
 - $somaAnterior[i \% 3] = dp[i]$ //fazemos isso depois para usar na próxima iteração...
- Em uma iteração, o valor que vamos somar depende do elemento que está na posição do vetor...
 - Exemplo, na iteração 1....
 - $somaAnterior[elem][]$ terá as somas anteriores considerando saltos de elem em elem..
 - $dp[i] = 1 + somaAnterior[elem][i \% elem]$

Se i= 9 e elem = 1, vamos somar:
Se i= 8 e elem = 1, vamos somar: 9
Se i= 7 e elem = 1, vamos somar: 8 9
Se i= 6 e elem = 1, vamos somar: 7 8 9
Se i= 5 e elem = 1, vamos somar: 6 7 8 9
Se i= 4 e elem = 1, vamos somar: 5 6 7 8 9
Se i= 3 e elem = 1, vamos somar: 4 5 6 7 8 9
Se i= 2 e elem = 1, vamos somar: 3 4 5 6 7 8 9
Se i= 1 e elem = 1, vamos somar: 2 3 4 5 6 7 8 9
Se i= 0 e elem = 1, vamos somar: 1 2 3 4 5 6 7 8 9

Se i= 9 e elem = 2, vamos somar:
Se i= 8 e elem = 2, vamos somar:
Se i= 7 e elem = 2, vamos somar: 9
Se i= 6 e elem = 2, vamos somar: 8
Se i= 5 e elem = 2, vamos somar: 7 9
Se i= 4 e elem = 2, vamos somar: 6 8
Se i= 3 e elem = 2, vamos somar: 5 7 9
Se i= 2 e elem = 2, vamos somar: 4 6 8
Se i= 1 e elem = 2, vamos somar: 3 5 7 9
Se i= 0 e elem = 2, vamos somar: 2 4 6 8

Se i= 9 e elem = 3, vamos somar:
Se i= 8 e elem = 3, vamos somar:
Se i= 7 e elem = 3, vamos somar:
Se i= 6 e elem = 3, vamos somar: 9
Se i= 5 e elem = 3, vamos somar: 8
Se i= 4 e elem = 3, vamos somar: 7
Se i= 3 e elem = 3, vamos somar: 6 9
Se i= 2 e elem = 3, vamos somar: 5 8
Se i= 1 e elem = 3, vamos somar: 4 7
Se i= 0 e elem = 3, vamos somar: 3 6 9

Se i= 9 e elem = 4, vamos somar:
Se i= 8 e elem = 4, vamos somar:
Se i= 7 e elem = 4, vamos somar:
Se i= 6 e elem = 4, vamos somar:
Se i= 5 e elem = 4, vamos somar: 9
Se i= 4 e elem = 4, vamos somar: 8
Se i= 3 e elem = 4, vamos somar: 7
Se i= 2 e elem = 4, vamos somar: 6
Se i= 1 e elem = 4, vamos somar: 5 9
Se i= 0 e elem = 4, vamos somar: 4 8

- Se elem for 3:
 - Temos que guardar 3 somatórios...
 - $dp[i] = 1 + somaAnterior[i \% 3]$
 - $somaAnterior[i \% 3] = dp[i]$ //fazemos isso depois para usar na próxima iteração...
- Em uma iteração, o valor que vamos somar depende do elemento que está na posição do vetor...
 - Exemplo, na iteração 1....
 - $somaAnterior[elem][]$ terá as somas anteriores considerando saltos de elem em elem..
 - $dp[i] = 1 + somaAnterior[elem][i \% elem]$
 - i. Exemplo: se elem = 1 → usamos apenas uma posição (0) (ou seja, sempre somamos o anterior)

Se i= 9 e elem = 1, vamos somar:
 Se i= 8 e elem = 1, vamos somar: 9
 Se i= 7 e elem = 1, vamos somar: 8 9
 Se i= 6 e elem = 1, vamos somar: 7 8 9
 Se i= 5 e elem = 1, vamos somar: 6 7 8 9
 Se i= 4 e elem = 1, vamos somar: 5 6 7 8 9
 Se i= 3 e elem = 1, vamos somar: 4 5 6 7 8 9
 Se i= 2 e elem = 1, vamos somar: 3 4 5 6 7 8 9
 Se i= 1 e elem = 1, vamos somar: 2 3 4 5 6 7 8 9
 Se i= 0 e elem = 1, vamos somar: 1 2 3 4 5 6 7 8 9

Se i= 9 e elem = 2, vamos somar:
 Se i= 8 e elem = 2, vamos somar:
 Se i= 7 e elem = 2, vamos somar: 9
 Se i= 6 e elem = 2, vamos somar: 8
 Se i= 5 e elem = 2, vamos somar: 7 9 //somaAnt[1]
 Se i= 4 e elem = 2, vamos somar: 6 8 //somaAnt[0]
 Se i= 3 e elem = 2, vamos somar: 5 7 9 //somaAnt[1]
 Se i= 2 e elem = 2, vamos somar: 4 6 8 //somaAnt[0]
 Se i= 1 e elem = 2, vamos somar: 3 5 7 9 //somaAnt[1]
 Se i= 0 e elem = 2, vamos somar: 2 4 6 8 //somaAnt[0]

Se i= 9 e elem = 3, vamos somar:
 Se i= 8 e elem = 3, vamos somar:
 Se i= 7 e elem = 3, vamos somar:
 Se i= 6 e elem = 3, vamos somar: 9
 Se i= 5 e elem = 3, vamos somar: 8
 Se i= 4 e elem = 3, vamos somar: 7
 Se i= 3 e elem = 3, vamos somar: 6 9
 Se i= 2 e elem = 3, vamos somar: 5 8
 Se i= 1 e elem = 3, vamos somar: 4 7
 Se i= 0 e elem = 3, vamos somar: 3 6 9

Se i= 9 e elem = 4, vamos somar:
 Se i= 8 e elem = 4, vamos somar:
 Se i= 7 e elem = 4, vamos somar:
 Se i= 6 e elem = 4, vamos somar:
 Se i= 5 e elem = 4, vamos somar: 9
 Se i= 4 e elem = 4, vamos somar: 8
 Se i= 3 e elem = 4, vamos somar: 7
 Se i= 2 e elem = 4, vamos somar: 6
 Se i= 1 e elem = 4, vamos somar: 5 9
 Se i= 0 e elem = 4, vamos somar: 4 8

- Se elem for 3:
 - Temos que guardar 3 somatórios...
 - $dp[i] = 1 + somaAnterior[i \% 3]$
 - $somaAnterior[i \% 3] = dp[i]$ //fazemos isso depois para usar na próxima iteração...
- Em uma iteração, o valor que vamos somar depende do elemento que está na posição do vetor...
 - Exemplo, na iteração 1....
 - $somaAnterior[elem][]$ terá as somas anteriores considerando saltos de elem em elem..
 - $dp[i] = 1 + somaAnterior[elem][i \% elem]$**
 - Exemplo: se elem = 2 → temos as somas 0 e a 1
 - $dp[3] = 1 + somaAnterior[2][1]$
 - $dp[2] = 1 + somaAnterior[2][0]$
 - $dp[1] = 1 + somaAnterior[2][1]$
 - $dp[0] = 1 + somaAnterior[2][0]$
 - Após calcular $dp[i]$, tenho que somá-lo a somaAnterior de todos elementos...

Se i= 9 e elem = 1, vamos somar:
 Se i= 8 e elem = 1, vamos somar: 9
 Se i= 7 e elem = 1, vamos somar: 8 9
 Se i= 6 e elem = 1, vamos somar: 7 8 9
 Se i= 5 e elem = 1, vamos somar: 6 7 8 9
Se i= 4 e elem = 1, vamos somar: 5 6 7 8 9
Se i= 3 e elem = 1, vamos somar: 4 5 6 7 8 9
 Se i= 2 e elem = 1, vamos somar: 3 4 5 6 7 8 9
 Se i= 1 e elem = 1, vamos somar: 2 3 4 5 6 7 8 9
 Se i= 0 e elem = 1, vamos somar: 1 2 3 4 5 6 7 8 9

Se i= 9 e elem = 2, vamos somar:
 Se i= 8 e elem = 2, vamos somar:
 Se i= 7 e elem = 2, vamos somar: 9
 Se i= 6 e elem = 2, vamos somar: 8
 Se i= 5 e elem = 2, vamos somar: 7 9
 Se i= 4 e elem = 2, vamos somar: 6 8
 Se i= 3 e elem = 2, vamos somar: 5 7 9
Se i= 2 e elem = 2, vamos somar: 4 6 8
 Se i= 1 e elem = 2, vamos somar: 3 5 7 9
 Se i= 0 e elem = 2, vamos somar: 2 4 6 8

Se i= 9 e elem = 3, vamos somar:
 Se i= 8 e elem = 3, vamos somar:
 Se i= 7 e elem = 3, vamos somar:
 Se i= 6 e elem = 3, vamos somar: 9
 Se i= 5 e elem = 3, vamos somar: 8
 Se i= 4 e elem = 3, vamos somar: 7
 Se i= 3 e elem = 3, vamos somar: 6 9
 Se i= 2 e elem = 3, vamos somar: 5 8
Se i= 1 e elem = 3, vamos somar: 4 7
 Se i= 0 e elem = 3, vamos somar: 3 6 9

Se i= 9 e elem = 4, vamos somar:
 Se i= 8 e elem = 4, vamos somar:
 Se i= 7 e elem = 4, vamos somar:
 Se i= 6 e elem = 4, vamos somar:
 Se i= 5 e elem = 4, vamos somar: 9
 Se i= 4 e elem = 4, vamos somar: 8
 Se i= 3 e elem = 4, vamos somar: 7
 Se i= 2 e elem = 4, vamos somar: 6
 Se i= 1 e elem = 4, vamos somar: 5 9
Se i= 0 e elem = 4, vamos somar: 4 8

- Se elem for 3:
 - Temos que guardar 3 somatórios...
 - $dp[i] = 1 + somaAnterior[i \% 3]$
 - $somaAnterior[i \% 3] = dp[i]$ //fazemos isso depois para usar na próxima iteração...
- Em uma iteração, o valor que vamos somar depende do elemento que está na posição do vetor...
 - Exemplo, na iteração 1....
 - $somaAnterior[elem][]$ terá as somas anteriores considerando saltos de elem em elem..
 - **$dp[i] = 1 + somaAnterior[elem][i \% elem]$**
 - i. Exemplo: se elem = 2 → temos as somas 0 e a 1
 - $dp[3] = 1 + somaAnterior[2][1]$
 - $dp[2] = 1 + somaAnterior[2][0]$
 - $dp[1] = 1 + somaAnterior[2][1]$
 - $dp[0] = 1 + somaAnterior[2][0]$
 - Após calcular $dp[i]$, tenho que somá-lo a somaAnterior de todos elementos...
 - i. Exemplo: calculei $dp[i=4]$, com $elem=v[4]=1$
 - Ele pode ser usado em $dp[3]$ (se $v[3]=1$)
 - Ele pode ser usado em $dp[2]$ (se $v[2]=2$)
 - Ele pode ser usado em $dp[1]$ (se $v[1]=3$)
 - ii. $somaAnterior[?][?] = dp[4]$

```

Se i= 9 e elem = 1, vamos somar:
Se i= 8 e elem = 1, vamos somar: 9
Se i= 7 e elem = 1, vamos somar: 8 9
Se i= 6 e elem = 1, vamos somar: 7 8 9 //usa somaAnterior[1][0]
Se i= 5 e elem = 1, vamos somar: 6 7 8 9 //usa somaAnterior[1][0]
Se i= 4 e elem = 1, vamos somar: 5 6 7 8 9 //usa somaAnterior[1][0]
Se i= 3 e elem = 1, vamos somar: 4 5 6 7 8 9 //usa somaAnterior[1][0]
Se i= 2 e elem = 1, vamos somar: 3 4 5 6 7 8 9 //usa somaAnterior[1][0]
Se i= 1 e elem = 1, vamos somar: 2 3 4 5 6 7 8 9 //usa somaAnterior[1][0]
Se i= 0 e elem = 1, vamos somar: 1 2 3 4 5 6 7 8 9 //usa somaAnterior[1][0]

```

```

Se i= 9 e elem = 2, vamos somar:
Se i= 8 e elem = 2, vamos somar:
Se i= 7 e elem = 2, vamos somar: 9
Se i= 6 e elem = 2, vamos somar: 8
Se i= 5 e elem = 2, vamos somar: 7 9 //usa somaAnterior[2][1]
Se i= 4 e elem = 2, vamos somar: 6 8 //usa somaAnterior[2][0]
Se i= 3 e elem = 2, vamos somar: 5 7 9 //usa somaAnterior[2][1]
Se i= 2 e elem = 2, vamos somar: 4 6 8 //usa somaAnterior[2][0]
Se i= 1 e elem = 2, vamos somar: 3 5 7 9 //usa somaAnterior[2][1]
Se i= 0 e elem = 2, vamos somar: 2 4 6 8 //usa somaAnterior[2][0]

```

```

Se i= 9 e elem = 3, vamos somar:
Se i= 8 e elem = 3, vamos somar:
Se i= 7 e elem = 3, vamos somar:
Se i= 6 e elem = 3, vamos somar: 9
Se i= 5 e elem = 3, vamos somar: 8 //usa somaAnterior[3][2]
Se i= 4 e elem = 3, vamos somar: 7 //usa somaAnterior[3][1]
Se i= 3 e elem = 3, vamos somar: 6 9 //usa somaAnterior[3][0]
Se i= 2 e elem = 3, vamos somar: 5 8 //usa somaAnterior[3][2]
Se i= 1 e elem = 3, vamos somar: 4 7 //usa somaAnterior[3][1]
Se i= 0 e elem = 3, vamos somar: 3 6 9 //usa somaAnterior[3][0]

```

```

Se i= 9 e elem = 4, vamos somar:
Se i= 8 e elem = 4, vamos somar:
Se i= 7 e elem = 4, vamos somar:
Se i= 6 e elem = 4, vamos somar:
Se i= 5 e elem = 4, vamos somar: 9 //usa somaAnterior[4][1]
Se i= 4 e elem = 4, vamos somar: 8 //usa somaAnterior[4][0]
Se i= 3 e elem = 4, vamos somar: 7 //usa somaAnterior[4][3]
Se i= 2 e elem = 4, vamos somar: 6 //usa somaAnterior[4][2]
Se i= 1 e elem = 4, vamos somar: 5 9 //usa somaAnterior[4][1]
Se i= 0 e elem = 4, vamos somar: 4 8 //usa somaAnterior[4][0]

```

- Se elem for 3:
 - Temos que guardar 3 somatórios...
 - $dp[i] = 1 + somaAnterior[i \% 3]$
 - $somaAnterior[i \% 3] = dp[i]$ //fazemos isso depois para usar na próxima iteração...
- Em uma iteração, o valor que vamos somar depende do elemento que está na posição do vetor...
 - Exemplo, na iteração 1....
 - $somaAnterior[elem][]$ terá as somas anteriores considerando saltos de elem em elem..
 - $dp[i] = 1 + somaAnterior[elem][i \% elem]$**
 - Exemplo: se elem = 2 → temos as somas 0 e a 1
 - $dp[3] = 1 + somaAnterior[2][1]$
 - $dp[2] = 1 + somaAnterior[2][0]$
 - $dp[1] = 1 + somaAnterior[2][1]$
 - $dp[0] = 1 + somaAnterior[2][0]$
 - Após calcular $dp[i]$, tenho que somá-lo a somaAnterior de todos elementos...
 - Exemplo: calculei $dp[i=4]$, com $elem=v[4]=1$
 - Ele pode ser usado em $dp[3]$ (se $v[3]=1$)
 - Ele pode ser usado em $dp[2]$ (se $v[2]=2$)
 - Ele pode ser usado em $dp[1]$ (se $v[1]=3$)
 - Ele pode ser usado em $dp[0]$ (se $v[1]=4$)
 - $somaAnterior[elem][i \% elem] = dp[i]$
- Qualquer elemento pode aparecer nas posições do array → temos que atualizar somaAnterior para todos elementos possíveis.

```
vector<ull> dp(n);

int maxElem = *max_element(v.begin(), v.end());
vector<vector<ull>> somaAnterior( 1 + maxElem , vector<ull>(maxElem,0));
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado
    //e que já calculei em dp as respostas para todos quadrados da frente
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;

    /*for(int x=1;i + x*elem < n;x++) { //salta de elem em elem a partir
        ans = (ans + dp[i + x*elem])%998244353;
    }*/
    ans = (ans + somaAnterior[elem][i%elem])%998244353;

    for(int elem2=1;elem2<=maxElem;elem2++) { //nao ha elemento 0 na entrada
        somaAnterior[ elem2 ][ i%elem2 ] += ans;
        somaAnterior[ elem2 ][ i%elem2 ] %= 998244353;
    }

    dp[i] = ans;
}
```

- Se elem for 3:
 - Temos que guardar 3 somatórios...
 - $dp[i] = 1 + somaAnterior[i \% 3]$
 - $somaAnterior[i \% 3] = dp[i]$ //fazemos isso depois para usar na próxima iteração...

Em uma iteração, o valor que vamos somar depende do elemento que está na posição do vetor...

- Exemplo, na iteração 1....
- $somaAnterior[elem][]$ terá as somas anteriores considerando saltos de elem em elem..
- $dp[i] = 1 + somaAnterior[elem][i \% elem]$**
 - Exemplo: se elem = 2 → temos as somas 0 e a 1
 - $dp[3] = 1 + somaAnterior[2][1]$
 - $dp[2] = 1 + somaAnterior[2][0]$
 - $dp[1] = 1 + somaAnterior[2][1]$
 - $dp[0] = 1 + somaAnterior[2][0]$
- Após calcular $dp[i]$, tenho que somá-lo a somaAnterior de todos elementos...
 - Exemplo: calculei $dp[i=4]$, com $elem=v[4]=1$
 - Ele pode ser usado em $dp[3]$ (se $v[3]=1$)
 - Ele pode ser usado em $dp[2]$ (se $v[2]=2$)
 - Ele pode ser usado em $dp[1]$ (se $v[1]=3$)
 - Ele pode ser usado em $dp[0]$ (se $v[1]=4$)
 - $somaAnterior[elem][i \% elem] = dp[4]$
- Qualquer elemento pode aparecer nas posições do array → temos que atualizar somaAnterior para todos elementos possíveis.

- Bem melhor!

```
vector<ull> dp(n);

int maxElem = *max_element(v.begin(), v.end());
vector<vector<ull> > somaAnterior( 1 + maxElem , vector<ull>(maxElem,0));
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado
    //e que já calculei em dp as respostas para todos quadrados da frente
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;

    /*for(int x=1;i + x*elem < n;x++) { //salta de elem em elem a partir
        ans = (ans + dp[i + x*elem])%998244353;
    }*/
    ans = (ans + somaAnterior[elem][i%elem])%998244353;

    for(int elem2=1;elem2<=maxElem;elem2++) { //nao ha elemento 0 na entrada
        somaAnterior[ elem2 ][ i%elem2 ] += ans;
        somaAnterior[ elem2 ][ i%elem2 ] %= 998244353;
    }

    dp[i] = ans;
}
```

- Bem melhor obter mais um TLE assim!

- Solução 1:

i. $O($

- Solução 2:

i. $O($

```
vector<ull> dp(n);

int maxElem = *max_element(v.begin(), v.end());
vector<vector<ull> > somaAnterior( 1 + maxElem , vector<ull>(maxElem,0));
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado i
    //e que já calculei em dp as respostas para todos quadrados da frente
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;

    /*for(int x=1;i + x*elem < n;x++) { //salta de elem em elem a partir de i
        ans = (ans + dp[i + x*elem])%998244353;
    }*/
    ans = (ans + somaAnterior[elem][i%elem])%998244353;

    for(int elem2=1;elem2<=maxElem;elem2++) { //nao ha elemento 0 na entrada
        somaAnterior[ elem2 ][ i%elem2 ] += ans;
        somaAnterior[ elem2 ][ i%elem2 ] %= 998244353;
    }

    dp[i] = ans;
}
```

```

vector<ull> dp(n);

int maxElem = *max_element(v.begin(), v.end());
vector<vector<ull> > somaAnterior( 1 + maxElem , vector<ull>(maxElem,0));
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado
    //e que já calculei em dp as respostas para todos quadrados da frente
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;

    /*for(int x=1;i + x*elem < n;x++) { //salta de elem em elem a partir
        ans = (ans + dp[i + x*elem])%998244353;
    }*/
    ans = (ans + somaAnterior[elem][i%elem])%998244353;

    for(int elem2=1;elem2<=maxElem;elem2++) { //nao ha elemento 0 na entrada
        somaAnterior[ elem2 ][ i%elem2 ] += ans;
        somaAnterior[ elem2 ][ i%elem2 ] %= 998244353;
    }

    dp[i] = ans;
}

```

- Bem melhor obter mais um TLE assim!
 - Solução 1:
 - i. $O(n * n / \text{Max})$
 - ii. Quanto maior o elemento, mais rápido $i + x * \text{elem}$ chega em n .
 - Solução 2:
 - i. $O($

```

vector<ull> dp(n);

int maxElem = *max_element(v.begin(), v.end());
vector<vector<ull> > somaAnterior( 1 + maxElem , vector<ull>(maxElem,0));
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado
    //e que já calculei em dp as respostas para todos quadrados da frente
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;

    /*for(int x=1;i + x*elem < n;x++) { //salta de elem em elem a partir
        ans = (ans + dp[i + x*elem])%998244353;
    }*/
    ans = (ans + somaAnterior[elem][i%elem])%998244353;

    for(int elem2=1;elem2<=maxElem;elem2++) { //nao ha elemento 0 na entrada
        somaAnterior[ elem2 ][ i%elem2 ] += ans;
        somaAnterior[ elem2 ][ i%elem2 ] %= 998244353;
    }

    dp[i] = ans;
}

```

- Bem melhor obter mais um TLE assim!
 - Solução 1:
 - i. $O(n * n / \text{MaxElem})$
 - ii. Quanto maior o elemento, mais rápido $i + x * \text{elem}$ chega em n .
 - Solução 2:
 - i. $O(n * \text{MaxElem})$
 - ii. Quanto maior maxElem , pior.
- Ou seja, se entrada tivesse apenas elementos grandes → usar solução 1.
- Se tiver apenas elementos pequenos → usar solução 2.
- Mas e se tiver grandes e pequenos?

```

vector<ull> dp(n);

int maxElem = *max_element(v.begin(), v.end());
vector<vector<ull> > somaAnterior( 1 + maxElem , vector<ull>(maxElem,0));
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado
    //e que já calculei em dp as respostas para todos quadrados da frente
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;

    /*for(int x=1;i + x*elem < n;x++) { //salta de elem em elem a partir
        ans = (ans + dp[i + x*elem])%998244353;
    }*/
    ans = (ans + somaAnterior[elem][i%elem])%998244353;

    for(int elem2=1;elem2<=maxElem;elem2++) { //nao ha elemento 0 na entrada
        somaAnterior[ elem2 ][ i%elem2 ] += ans;
        somaAnterior[ elem2 ][ i%elem2 ] %= 998244353;
    }

    dp[i] = ans;
}

```

- Bem melhor obter mais um TLE assim!
 - Solução 1:
 - i. $O(n * n / \text{MaxElem})$
 - ii. Quanto maior o elemento, mais rápido $i + x * \text{elem}$ chega em n .
 - Solução 2:
 - i. $O(n * \text{MaxElem})$
 - ii. Quanto maior maxElem , pior.
- Ou seja, se entrada tivesse apenas elementos grandes → usar solução 1.
- Se tiver apenas elementos pequenos → usar solução 2.
- Mas e se tiver grandes e pequenos?
 - Colocar um if dentro do for: se elemento for $< \sqrt{n}$ → calcular a iteração com solução 2.
 - “somaAnterior” vai guardar somas apenas para valores pequenos.
 - Se não, calcular a iteração com solução 1.
 - Qual será a complexidade?

```

vector<ull> dp(n);

int maxElem = *max_element(v.begin(), v.end());
vector<vector<ull> > somaAnterior( 1 + maxElem , vector<ull>(maxElem,0));
for(int i=n-1;i>=0;i--) {
    //de quantas formas consigo pintar a partir de i, supondo que o quadrado
    //e que já calculei em dp as respostas para todos quadrados da frente
    int elem = v[i]; // elemento que está na posicao i

    ull ans = 1;

    /*for(int x=1;i + x*elem < n;x++) { //salta de elem em elem a partir
        ans = (ans + dp[i + x*elem])%998244353;
    }*/
    ans = (ans + somaAnterior[elem][i%elem])%998244353;

    for(int elem2=1;elem2<=maxElem;elem2++) { //nao ha elemento 0 na entrada
        somaAnterior[ elem2 ][ i%elem2 ] += ans;
        somaAnterior[ elem2 ][ i%elem2 ] %= 998244353;
    }

    dp[i] = ans;
}

```

- Bem melhor obter mais um TLE assim!
 - Solução 1:
 - i. $O(n * n / \text{MaxElem})$
 - ii. Quanto maior o elemento, mais rápido $i + x * \text{elem}$ chega em n .
 - Solução 2:
 - i. $O(n * \text{MaxElem})$
 - ii. Quanto maior maxElem , pior.
- Ou seja, se entrada tivesse apenas elementos grandes → usar solução 1.
- Se tiver apenas elementos pequenos → usar solução 2.
- Mas e se tiver grandes e pequenos?
 - Colocar um if dentro do for: se elemento for $< \sqrt{n}$ → calcular a iteração com solução 2.
 - “somaAnterior” vai guardar somas apenas para valores pequenos.
 - Se não, calcular a iteração com solução 1.
 - Qual será a complexidade? $O(n \sqrt{n})$ sempre!

SQRT Decomposition

- Exercício: resolva o problema, juntando as duas soluções.

Algoritmo de Mo

- Array de números + consultas de intervalo para encontrar a média de cada intervalo.
- 30k elementos, 200k consultas, valores até 1M
- Como resolver?

```
5
1 1 2 1 3
3
1 5
2 4
3 5
```

Algoritmo de Mo

- Array de números + consultas de intervalo para encontrar a média de cada intervalo.
- 30k elementos, 200k consultas, valores até 1M
- Como resolver?
 - Prefix-sum (porém, Mo resolve problemas que prefix não resolveria)
 - Seg-tree, etc.

```
5
1 1 2 1 3
3
1 5
2 4
3 5
```

Algoritmo de Mo

- Algoritmo de Mo: (nome vem de um competidor chines)
 - Usado para consultas eficientes em um array ESTÁTICO.
 - Como array é estático, as consultas podem ser processadas em qualquer ordem.
 - Algoritmo reorganiza consultas de modo que elas possam ser calculadas de forma eficiente.
 - i. É mantido um range ativo do array. A qualquer momento sabemos o valor da consulta para esse range.
 - ii. Consultas são processadas uma a uma em uma ordem eficiente.
 - iii. Complexidade: $O(f(n) n \sqrt{n})$, onde $f(n)$ é o custo de se inserir/remover elementos.

```
5
1 1 2 1 3
3
1 5
2 4
3 5
```

Algoritmo de Mo

- Para esse problema:
 - Dados do range serão guardados em um array, além da soma e da quantidade de elementos.
 - Se range ativo for o abaixo, soma = 7 , n = 4
 - Se left andar 1 para direita e right andar 2 → soma = 16, n = 5

0	3	2	1	1	3	5	6	3	1
---	---	---	---	---	---	---	---	---	---

0	3	2	1	1	3	5	6	3	1
---	---	---	---	---	---	---	---	---	---

Algoritmo de Mo

- Veja o código mo_algorithm.cpp, que resolve o problema.
- A classe abaixo mantém a estatística (consulta) sobre a janela.

```
//adapte essa classe!  
class JanelaMo {  
public:  
    JanelaMo(vector<int> &v_): v(v_) { //tamanho do array  
        soma = 0;  
        n = 0;  
    }  
    double valorConsulta() { //adaptar: retorna o valor da consulta da janela atual  
        return ((double)soma)/n;  
    }  
    void add(int idx) { //adiciona o elemento da posicao idx a janela  
        soma += v[idx];  
        n++;  
    }  
    void remove(int idx) {  
        soma -= v[idx];  
        n--;  
    }  
  
private:  
    vector<int> &v; //array de entrada (estático!) . Adaptar para outros tipos...  
    int soma;  
    int n;  
};
```

Algoritmo de Mo

- Para esse problema:
 - `mo_left` : posição esquerdo da janela (range)
 - `mo_right`: posição direita da janela
- Para responder a uma consulta (`left, right`):
 - Basta andar com (`mo_left`, `mo_right`) até (`left`, `right`), atualizando o valor da janela.

```
JanelaMo janela(v);

int mo_left = -1;
int mo_right = -1;
vector<double> ans(q); //respostas das consultas (adaptar)
for (int i = 0; i < q; i++) {
    int left = queries[i].l;
    int right = queries[i].r;

    if(mo_left==-1) { //primeira consulta...
        for(int i=left; i<=right; i++) janela.add(i);
        mo_left = left;
        mo_right = right;
    } else {
        while (mo_left < left) { janela.remove(mo_left++); } //
        while (mo_left > left) { janela.add(--mo_left); }
        while (mo_right < right) { janela.add(++mo_right); }
        while (mo_right > right) { janela.remove(mo_right--); }
    }

    ans[queries[i].idx] = janela.valorConsulta() ;
}
```

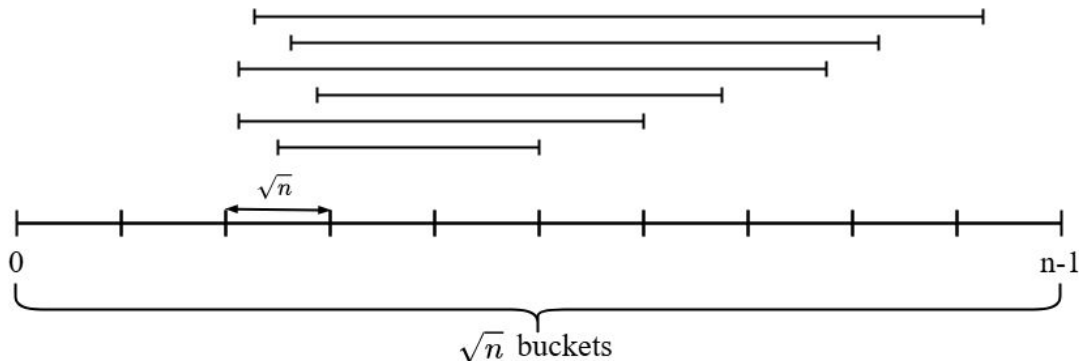
Algoritmo de Mo

- Até aqui, tudo é muito óbvio.
- O segredo é a ORDEM do processamento das consultas.
 - Array é dividido em blocos de tamanho $\sqrt{N}$.
 - Consultas ordenadas assim $q_a < q_b$, se:
 - i. $\text{bloco}(q_a.\text{left})$ vem antes de $\text{bloco}(q_b.\text{left})$
 - ii. Os left estão no mesmo bloco, mas $q_a.\text{right} < q_b.\text{right}$

```
int block_size = (int)sqrt(n);
auto mo_cmp = [&](Query a, Query b) {
    int block_a = a.l / block_size;
    int block_b = b.l / block_size;
    if (block_a == block_b) { return a.r < b.r; }
    return block_a < block_b;
};
sort(queries.begin(), queries.end(), mo_cmp);
```

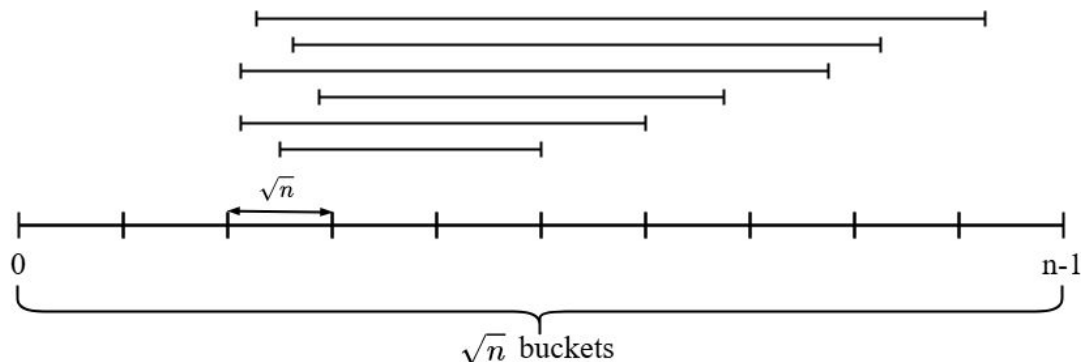
Algoritmo de Mo

- O segredo é a ORDEM do processamento das consultas.
 - Array é dividido em blocos de tamanho $\sqrt{N}$.
 - Consultas ordenadas assim $q_a < q_b$, se:
 - i. $\text{bloco}(q_a.\text{left})$ vem antes de $\text{bloco}(q_b.\text{left})$
 - ii. Os left estão no mesmo bloco, mas $q_a.\text{right} < q_b.\text{right}$
 - Ou seja, em cada passo você processa um dos blocos.
 - Ao processar um bloco, temos os elementos com right em ordem crescente (o left fica “oscliando” entre 0 e $\sqrt{N}$ dentro do bloco)



Algoritmo de Mo

- O segredo é a ORDEM do processamento das consultas.
 - Ou seja, em cada passo você processa um dos blocos.
 - Ao processar um bloco, temos os elementos com right em ordem crescente (o left fica “oscilando” entre 0 e $\sqrt{N}$ dentro do bloco)
 - i. Complexidade fica $O(f(n) N \sqrt{N})$, onde $f(n)$ é o custo das funções add e remove.
 - ii. O left dos blocos varia $\sqrt{N}$ posições N vezes $\rightarrow N * \sqrt{N}$
 - iii. O right dos blocos anda no máximo N posições $\sqrt{N}$ vezes (uma por bloco) $\rightarrow \sqrt{N} * N$
 - Pois o right anda no máximo de 0 até o final (N posições), uma única vez em cada bloco.
 - Há $\sqrt{N}$ blocos $\rightarrow \sqrt{N}$ vezes



Algoritmo de Mo

- Código do USACO tinha um bug:
 - Passava em problemas da USACO.
 - Dava RTE em um do CF. Corrigi o bug e coloquei no github... cuidado com código da internet!
- Em geral, o mais importante será adaptar a classe “JanelaMo” para atualizar as estatísticas da janela atual.
- Bom para $n = \sim 10^4$. Para 10^5 talvez (talvez calibrar tamanho dos blocos!)
- Pode ser útil se for difícil implementar a função de juntar segmentos em uma seg tree. Exemplo: calcular moda, quantos elementos únicos há em um intervalo, etc.
- Talvez seja mais “seguro” adicionar os elementos antes de remover à medida em que a janela se move. Recomendo que deixem na sua biblioteca um código que faça assim. Exemplo:
https://cp-algorithms.com/data_structures/sqrt_decomposition.html
- Tem problemas que envolvem Mo em uma árvore. Árvore é linearizada (HLD, euler) e Mo é utilizado.
- SQRT e Mo são ideias “gerais”: importante resolver vários problemas para se acostumar com a ideia e identificar aplicações.
- Há uma otimização que usa curvas de Hilbert para acelerar o algoritmo de MO. Pode resolver algum TLE:
<https://codeforces.com/blog/entry/61203> (exemplo: distinct colors dava TLE sem essa otimização!)
- Há uma versão de Mo com atualizações:
<https://www.geeksforgeeks.org/dsa/mos-algo-with-update-and-without-update/> ,
<https://www.youtube.com/watch?v=gUpfwVRXhNY>
- Como exercício, adapte o código fornecido para resolver o problema <https://www.spoj.com/problems/DQUERY/>